

I. WHY TRADITIONAL NETWORKS FAIL AT IMAGES

In File 16, we discussed Multi-Layer Perceptrons (MLP). If we pass a 256 x 256 color image to an MLP, the input layer would need 196,608 neurons ($256 \times 256 \times 3$ channels). This leads to:

1. **Parameter Explosion:** Too many weights to train, causing the model to crash or overfit.
2. **Loss of Spatial Features:** MLPs "flatten" the image into a long string of numbers, losing the relationship between neighboring pixels (e.g., an eye is next to a nose).

CNNs solve this by using **Convolution**, which preserves the spatial structure of the data.

II. THE ARCHITECTURE OF A CNN

A CNN consists of specialized layers that act as "filters" to extract features from an image.

2.1. The Convolutional Layer (The Filter)

A small matrix (e.g., 3×3), called a **Kernel** or **Filter**, slides across the image. It performs a mathematical operation to detect specific patterns.

- **Early Layers:** Detect simple edges, lines, and colors.
- **Deep Layers:** Detect complex shapes, textures, and eventually whole objects (faces, cars).

2.2. The Activation Function (ReLU)

After convolution, we apply **ReLU** to remove negative pixel values, ensuring the model only focuses on "positive" features.

2.3. The Pooling Layer (Downsampling)

Pooling reduces the size of the image to make the computation faster and the model more robust.

- **Max Pooling:** Takes the maximum value from a small window (e.g., 2×2). It effectively says: "If a feature was detected anywhere in this area, keep the strongest signal."

2.4. Fully Connected (FC) Layer

At the very end, the extracted features are flattened and passed to a standard MLP to make the final classification (e.g., "This is a Cat").

III. KEY HYPERPARAMETERS IN CNNs

- **Stride:** The number of pixels the filter moves at each step. A higher stride means a smaller output image.
- **Padding:** Adding "fake" pixels (usually zeros) around the edge of an image so the filter can cover the corners without shrinking the output.
- **Channels:** A standard image has 3 (Red, Green, Blue). A CNN layer might have 64 or 128 channels, each looking for a different feature.

IV. PRACTICAL LAB: BUILDING A CNN (PYTORCH)

This code demonstrates a standard CNN structure used for classifying images like the CIFAR-10 dataset.

Python

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
class SimpleCNN(nn.Module):
```

```
    def __init__(self):
```

```
        super(SimpleCNN, self).__init__()
```

```
        # Input: 3 channels (RGB). Output: 16 filters. Kernel size: 3x3.
```

```
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2) # Reduces image size by half
```

```
        # Second layer: 16 input channels -> 32 output filters
```

```
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
```

```
        # Final classification layers
```

```
        self.fc1 = nn.Linear(32 * 8 * 8, 128) # Assuming 32x32 input reduced to 8x8
```

```
        self.fc2 = nn.Linear(128, 10) # 10 classes
```

```
    def forward(self, x):
```

```
        x = self.pool(F.relu(self.conv1(x)))
```

```
        x = self.pool(F.relu(self.conv2(x)))
```

```
        x = x.view(-1, 32 * 8 * 8) # Flatten
```

```
        x = F.relu(self.fc1(x))
```

```
        x = self.fc2(x)
```

```
        return x
```

V. FAMOUS CNN ARCHITECTURES

- **LeNet-5 (1998):** Used for handwritten digit recognition (the "Grandfather" of CNNs).
- **AlexNet (2012):** The model that started the modern AI revolution by winning the ImageNet competition.
- **ResNet (Residual Networks):** Introduced "Skip Connections" to allow training of networks with hundreds of layers without losing information.